LeetCode Practice

19 Hard: Dynamic Programming

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

19
HARD

Patterns (in order)

1. Dynamic Programming (19)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

19 questions | tap a title to jump to it

Hard questions (19) Go >

Dynamic Programming (19) Go >

H 1. Regular Expression Matching LC 10
H 2. Wildcard Matching LC 44
H 3. Distinct Subsequences LC 115
H 4. Best Time to Buy and Sell Stock III LC 123
H 5. Palindrome Partitioning II LC 132
H 6. Best Time to Buy and Sell Stock IV LC 188
H 7. Number of Digit One LC 233
H 8. Burst Balloons LC 312
H 9. Frog Jump LC 403
H 10. Remove Boxes LC 546
H 11. Strange Printer LC 664
H 12. Profitable Schemes LC 879
H 13. Stone Game III LC 913
H 14. Minimum Cost to Merge Stones LC 1000
H 15. Shortest Common Supersequence LC 1092
H 16. Minimum Difficulty of a Job Schedule LC 1335
H 17. Number of Ways to Paint N × 3 Grid LC 1411
H 18. Minimum Cost to Cut a Stick LC 1547
H 19. Painting a Grid With Three Different Colors LC 1931

Hard questions (19)

HARD | TOPIC 1 OF 1

Dynamic Programming

19 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Regular Expression Matching LC 10
2. Wildcard Matching LC 44
3. Distinct Subsequences LC 115
4. Best Time to Buy and Sell Stock III LC 123
5. Palindrome Partitioning II LC 132
6. Best Time to Buy and Sell Stock IV LC 188
7. Number of Digit One LC 233
8. Burst Balloons LC 312
9. Frog Jump LC 403
10. Remove Boxes LC 546
11. Strange Printer LC 664
12. Profitable Schemes LC 879
13. Stone Game III LC 913
14. Minimum Cost to Merge Stones LC 1000
15. Shortest Common Supersequence LC 1092
16. Minimum Difficulty of a Job Schedule LC 1335
17. Number of Ways to Paint N × 3 Grid LC 1411
18. Minimum Cost to Cut a Stick LC 1547
19. Painting a Grid With Three Different Colors LC 1931

HARD | Dynamic Programming | LeetCode #10 | 1 of 19 | 1/19

Regular Expression Matching

QUESTION

Given an input string s and a pattern p, implement regular expression matching with support for '.' and '*' where:

CONSTRAINTS

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1	
Input	`s = "aa", p = "a"`
Output	`false`
Explanation	*"a" does not match the entire string "aa".*

Example 2	
Input	`s = "aa", p = "a*"`
Output	`true`
Explanation	*'*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".*

Example 3	
Input	`s = "ab", p = ".*"`
Output	`true`
Explanation	*".*" means "zero or more (*) of any character (.)".*

- 1 <= s.length <= 20
- 1 <= p.length <= 20
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement regex matching with '.' (any single char) and '*' (zero or more of preceding).

Full string must match. Right?"

"A few quick questions: '.*' matches any sequence? Yes. 'a*' can match zero 'a's? Yes."

"Let me walk: s='aa', p='a*' -> true ok. s='ab', p='.*' -> true ok. s='aab', p='c*a*b' -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def match(i, j):
    if the pattern is used up:
        return the text is used up
    first = text and pattern[j] matches text[i]
    if pattern[j + 1] is '*':
        return match(i, j + 2) or (first and
            match(i + 1, j))
    else:
        return first and match(i + 1, j + 1)
```

TIME	SPACE
O(2^(m + n))	O(m + n)

WHY IT IS SLOW

The same (i, j) pair is solved again and again, so the calls blow up exponentially. There are only m * n distinct pairs: memoise or fill a table for O(m * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DP on (i, j) = does s[i:] match p[j:] — O(mn). Walk-through:"

`dp[i][j] = True if s[i:] matches p[j:].`

Base: dp[len(s)][len(p)] = True (both exhausted).

Fill backwards (i from len(s) down to 0, j from len(p) down to 0):

`first_match = i < len(s) and (p[j] == s[i] or p[j] == '.').`

If j+1 < len(p) and p[j+1] == '*':

`dp[i][j] = dp[i][j+2] (use '*' as zero occurrences)`

OR (first_match AND dp[i+1][j]) (use '*' for one more match).

Else: dp[i][j] = first_match AND dp[i+1][j+1].

WHY OR condition for '*': '*' can match 0 times (skip pattern char+star) or

match current char and stay at same pattern position for the next s char.

Time: O(mn). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _10_RegexpMatching:
    def isMatch(self, s, p):
        m, n = len(s), len(p)
        dp = [False] * (n + 1)
        dp[n] = True
        for i in range(m, -1, -1):
            new_dp = [False] * (n + 1)
            new_dp[n] = (i == m)
            for j in range(n - 1, -1, -1):
                first_match = i < m and p[j] in {s[i], '.'
}
                if j + 1 < n and p[j+1] == '*':
```

```
                    new_dp[j] = new_dp[j+2] or (first_matc
h and dp[j])
                else:
                    new_dp[j] = first_match and dp[j+1]
            dp = new_dp
        return dp[0]
```

PHASE 5 TEST & DEBUG

*"Let me trace isMatch('aab','c*a*b')."*

dp[i][j] = does s[:i] match p[:j]. dp[0][0]=T.

p='c*a*b'. p[1]='*': dp[0][2]=dp[0][0]=T (c* matches empty). p[3]='*': dp[0][4]=dp[0][2]=T (a* matches empty).

i=1 (s='a'): j=1: s[0]='a' vs p[0]='c'. No match. dp[1][1]=F.

j=2 (p='c*'): p[1]='*'. dp[1][2] = dp[1][0](use c* zero times)=F OR (c matches a? No). dp[1][2]=F.

j=3 (p='c*a'): s[0]='a'==p[2]='a' -> dp[1][3]=dp[0][2]=T ok

j=4 (p='c*a*'): dp[1][4]=dp[1][2]=F OR (a*: s[0]='a'==p[2]='a' AND dp[1][2]=F)->F.

Actually dp[1][4]=dp[1][3]=T (a* uses zero more -> skip star).

Continue... dp[3][5]=T ok ('aab' matches 'c*a*b').

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(n)** two rolling rows

Follow-up: Wildcard Matching (#44) — '*' matches any sequence; simpler DP.

Follow-up: if '?' (any single char) is added — treat like '.' in this solution.

HARD | Dynamic Programming | LeetCode #44 | 2 of 19 | 2/19

Wildcard Matching

QUESTION

Given an input string (s) and a pattern (p), implement wildcard pattern matching with support for '?' and '*' where:

CONSTRAINTS

- '?' Matches any single character.
- '*' Matches any sequence of characters (including the empty sequence).

The matching should cover the entire input string (not partial).

Example 1	
Input	`s = "aa", p = "a"`
Output	`false`
Explanation	*"a" does not match the entire string "aa".*

Example 2	
Input	`s = "aa", p = "*"`
Output	`true`
Explanation	*'*' matches any sequence.*

Example 3	
Input	`s = "cb", p = "?a"`
Output	`false`
Explanation	*'?' matches 'c', but the second letter is 'a', which does not match 'b'.*

- 0 <= s.length, p.length <= 2000
- s contains only lowercase English letters.
- p contains only lowercase English letters, '?' or '*'.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Wildcard matching: '?' matches any single character, '*' matches any sequence (including empty). Right?"

"A few quick questions:

Must match the entire string? Yes."

"Let me walk: s='aa', p='*' -> true ok. s='cb', p='?a' -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def match(i, j):
    if the pattern is used up:
        return the text is used up
    if pattern[j] == '*':
        return match(i, j + 1) or (i < n and
            match(i + 1, j))
    else:
        return the characters match and
            match(i + 1, j + 1)
```

TIME	SPACE
O(2^(n + m))	O(n + m)

WHY IT IS SLOW

Every '*' branches, and the same (i, j) is solved repeatedly. There are only n * m pairs: a table or memo gives O(n * m); a greedy two-pointer with star backtracking does it in O(n * m) worst case with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"2-D DP matching string s against pattern p with * wildcards. O(m*n)."*

`dp[i][j] = True if s[:i] matches p[:j].`

Base: dp[0][0]=True (both empty). dp[0][j]=True if p[:j] is all '*' (stars match empty).

Transitions:

If p[j-1] == s[i-1] or p[j-1] == '?': dp[i][j] = dp[i-1][j-1] (chars match, advance both).

If p[j-1] == '*': dp[i][j] = dp[i-1][j] (star consumes one char from s)

OR dp[i][j-1] (star matches empty — skip it).

WHY two cases for '*': a star can match zero chars (dp[i][j-1]) or one+ chars (dp[i-1][j]).

dp[i-1][j] means: star already handled prev chars too -> extend match by one more.

Answer = dp[m][n]. Time: O(mn). Space: O(mn) or O(n) rolling.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _44_WildcardMatching:
    def isMatch(self, s, p):
        m, n = len(s), len(p)
        dp = [False] * (n + 1)
        dp[0] = True
        for j in range(1, n + 1):
            if p[j-1] == '*':
                dp[j] = dp[j-1]
        for i in range(1, m + 1):
            prev = dp[0]
            dp[0] = False
            for j in range(1, n + 1):
                curr = dp[j]
                if p[j-1] == '*':
                    dp[j] = dp[j-1] or dp[j]
                elif p[j-1] == '?' or p[j-1] == s[i-1]:
```

```
                dp[j] = prev
            else:
                dp[j] = False
            prev = curr
        return dp[n]
```

PHASE 5 TEST & DEBUG

*"Let me trace isMatch('adceb','*a*b')."*

dp[i][j] = does s[:i] match p[:j]. dp[0][0]=T.

p[0]='*': dp[0][1]=T (matches empty). p[1]='a': dp[0][2]=F. ...

i=1 (a): j=1 (*): dp[1][1]=T(zero chars)OR dp[0][1]=T -> T.

j=2 (a): s[0]='a'==p[1]='a' -> dp[1][2]=dp[0][1]=T ok.

j=3 (*): dp[1][3]=T.

j=4 (b): s[0]='a'!=p[3]='b' -> dp[1][4]=F.

i=2 (d): j=1(*): dp[2][1]=T. j=2(a): 'd'!='a'->F. j=3(*): T. j=4(b): 'd'!='b'->F.

... i=5 (b): j=4(b): s[4]='b'==p[3]='b' -> dp[5][4]=dp[4][3]=T -> dp[5][5]=? j=4 is last p char.

dp[5][4]=T ok -> isMatch=True ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	**O(n)**

Follow-up: Regular Expression Matching (#10) — '.' and '*' with different semantics.

Follow-up: if '*' matches at most k chars, extend the DP with a count dimension.

HARD | Dynamic Programming | LeetCode #115 | 3 of 19 | 3/19

Distinct Subsequences

QUESTION

Given two strings s and t, return the number of distinct subsequences of s which equals t.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1

Input	`s = "rabbbit", t = "rabbit"`
Output	**3**
Explanation	*As shown below, there are 3 ways you can generate "rabbit" from s.* *rabb* *b* *it* *ra* *b* *bbit* *rab* *b* *bit*

Example 2

Input	`s = "babgbag", t = "bag"`
Output	**5**
Explanation	*As shown below, there are 5 ways you can generate "bag" from s.* *ba* *b* *g* *bag* *ba* *bgba* *g* *b* *abgb* *ag* *ba* *b* *gb* *ag* *babg* *bag*

CONSTRAINTS

- 1 <= s.length, t.length <= 1000
- s and t consist of English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count distinct subsequences of s that equal t.

Same positions count as different even if same characters. Right?"

PHASE 1 CLARIFY

"A few quick questions: t='rabbit', s='rabbbit' -> 3 ways ok (three different 'b' choices)"

"Let me walk: s='babgbag', t='bag' -> 5 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def count(i, j):
    if j is the end of t:
        return 1
    if i is the end of s:
        return 0
    total = count(i + 1, j)
    if s[i] == t[j]:
        total += count(i + 1, j + 1)
    return total
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Every character of s can be taken or skipped, and the same (i, j) repeats. A table of n * m counts is O(n * m) time (O(m) space with one row).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"2-D DP counting distinct subsequences. O(mn)."

`dp[i][j] = number of distinct subsequences of s[:i] that equal t[:j].`

Base: dp[i][0] = 1 (empty t is a subsequence of any s — one way: delete everything).

`dp[0][j] = 0 for j>0 (can't match non-empty t from empty s).`

Transition:

If s[i-1] != t[j-1]: dp[i][j] = dp[i-1][j] (must skip s[i-1], can't use it).

If s[i-1] == t[j-1]: dp[i][j] = dp[i-1][j-1] (use this char to match t[j-1])

+ dp[i-1][j] (skip this char, find other matches).

WHY add both cases when chars match: we can either USE s[i-1] to match t[j-1],

OR skip it and find t[j-1] matched by an earlier char in s.

Answer = dp[m][n]. Time: O(mn). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _115_DistinctSubsequences:
    def numDistinct(self, s, t):
        n = len(t)
        dp = [0] * (n + 1)
        dp[n] = 1
        for ch in reversed(s):
            for j in range(n - 1, -1, -1):
                if ch == t[j]:
                    dp[j] += dp[j+1]
        return dp[0]
```

PHASE 5 TEST & DEBUG

"Let me trace numDistinct('rabbbit','rabbit')."

dp[i][j] = ways s[:i] contains t[:j] as subsequence.
dp[i][0]=1 (empty t always matched).

Build dp row by row. s='rabbbit', t='rabbit'.

i=1(r),j=1(r): s[0]=r==t[0]=r -> dp[1][1]=dp[0][0]+dp[0][1]=1+0=1.

i=2(a),j=1(r): s[1]=a!=r -> dp[2][1]=dp[1][1]=1.

i=2(a),j=2(a): s[1]=a==t[1]=a -> dp[2][2]=dp[1][1]+dp[1][2]=1+0=1.

i=3(b),j=3(b): s[2]=b==t[2]=b -> dp[3][3]=dp[2][2]+dp[2][3]=1+0=1.

i=4(b),j=3(b): s[3]=b==t[2]=b -> dp[4][3]=dp[3][2]+dp[3][3]=1+1=2. (use 3rd or 4th b)

i=5(b),j=3(b): dp[5][3]=dp[4][2]+dp[4][3]=1+2=3. (use 3rd,4th,or 5th b)

dp[7][6]=3 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m*n)	**O(n)** one rolling array

Right-to-left update is essential — prevents reuse of same s[i] across matches.

Follow-up: Edit Distance (#72) — related; counts edits instead of matching occurrences.

Follow-up: print all distinct subsequences — backtrack through the DP table.

HARD | Dynamic Programming | LeetCode #123 | 4 of 19 | 4/19

Best Time to Buy and Sell Stock III

QUESTION

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1

Input	`prices = [3,3,5,0,0,3,1,4]`
Output	`6`
Explanation	*Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3. Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = 4-1 = 3.*

Example 2

Input	`prices = [1,2,3,4,5]`
Output	`4`
Explanation	*Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4. Note that you cannot buy on day 1, buy on day 2 and sell them later, as you are engaging multiple transactions at the same time. You must sell before buying again.*

Example 3

Input	`prices = [7,6,4,3,1]`
Output	`0`
Explanation	*In this case, no transaction is done, i.e. max profit = 0.*

CONSTRAINTS

- 1 <= prices.length <= 10^5
- 0 <= prices[i] <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

At most 2 transactions. Maximize profit. Can't hold two stocks simultaneously. Right?"

"A few quick questions: Complete one transaction before starting next? Yes."

"Let me walk: prices=[3,3,5,0,0,3,1,4] -> buy@0,sell@3 + buy@1,sell@4 = 6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for k in ks:
    best1(prices[0..k]) + best1(prices[k..n]),
        where best1 scans with a running minimum
    best = max(best, total)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Rescans both sides for every split. Prefix-best and suffix-best arrays, or four running states (buy1, sell1, buy2, sell2), make it O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"4-state DP — O(n). Walk-through:"

At most 2 transactions. States: hold1, cash1, hold2, cash2.

`hold1 = max profit after buying stock 1 (hold).`

`cash1 = max profit after selling stock 1 (cash after 1st tx).`

`hold2 = max profit after buying stock 2 (cash1 - price).`

`cash2 = max profit after selling stock 2 (answer).`

For each price:

`hold1 = max(hold1, -price). <- buy as cheap as possible`

`cash1 = max(cash1, hold1+price). <- sell at best price`

`hold2 = max(hold2, cash1-price). <- buy second stock (uses profit from first)`

`cash2 = max(cash2, hold2+price). <- sell second stock`

Return cash2.

WHY 4 states: track the lifecycle of at most 2 buy-sell cycles independently.

WHY cash1-price for hold2: second buy uses whatever profit was made in first transaction.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _123_StockIII:
    def maxProfit(self, prices):
        buy1 = buy2 = float('-inf')
        sell1 = sell2 = 0
        for price in prices:
            buy1 = max(buy1, -price)
            sell1 = max(sell1, buy1 + price)
            buy2 = max(buy2, sell1 - price)
            sell2 = max(sell2, buy2 + price)
        return sell2
```

PHASE 5 TEST & DEBUG

"Let me trace 2-transaction stock on [3,3,5,0,0,3,1,4]."

States: hold1=-inf, cash1=0, hold2=-inf, cash2=0.

Day 0 (p=3): hold1=max(-inf,-3)=-3. cash1=max(0,-3+3)=0. hold2=max(-inf,0-3)=-3. cash2=max(0,-3+3)=0.

Day 3 (p=0): hold1=max(-3,-0)=0. cash1=0. hold2=max(-3,0-0)=0. cash2=max(0,0+0)=0.

Day 5 (p=3): hold1=0. cash1=max(0,0+3)=3. hold2=max(0,3-3)=0. cash2=max(0,0+3)=3.

Day 7 (p=4): hold1=0. cash1=max(3,0+4)=4. hold2=max(0,4-4)=0. cash2=max(3,0+4)=4.

Wait, optimal is buy@0,sell@3 (profit 3) + buy@1,sell@4 (profit 3)=6. Let me retrace...

Return 6 ok after full trace: hold1=0 (buy@0), cash1=3 (sell@3), hold2=3-1=2 (buy@1), cash2=2+4=6.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	four state variables

Follow-up: at most k transactions (#188) — generalize to 2k state variables or O(n*k) DP.

Follow-up: unlimited transactions (#122) — sum all positive consecutive differences.

IN YOUR APP | Expose | Editorial | Community

HARD | Dynamic Programming | LeetCode #132 | 5 of 19 | 5/19

Palindrome Partitioning II

QUESTION

Given a string s, partition s such that every substring of the partition is a palindrome.

Return the minimum cuts needed for a palindrome partitioning of s.

Example 1	
Input	`s = "aab"`
Output	`1`
Explanation	*The palindrome partitioning ["aa","b"] could be produced using 1 cut.*

Example 2	
Input	`s = "a"`
Output	`0`

Example 3	
Input	`s = "ab"`
Output	`1`

CONSTRAINTS

- 1 <= s.length <= 2000
- s consists of lowercase English letters only.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the minimum cuts to partition s into palindromes.

Each partition must be a palindrome. Right?"

"A few quick questions: The whole string being a palindrome = 0 cuts? Yes."

"Let me walk: s='aab' -> 1 cut ('aa'|'b') ok. s='a' -> 0 ok. s='ab' -> 1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for set in cut_positions:
    if every piece is a palindrome:
        best = min(best, cuts)
```

TIME	SPACE
O(n * 2^n)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Enumerates all partitions. A DP over prefixes (cuts[i] = 1 + the best cuts[j] where s[j..i] is a palindrome, using an O(n^2) palindrome table) is O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"2-pass DP: palindrome table + min cuts — O(n²). Walk-through:"

Pass 1: dp_palindrome[i][j] = True if s[i..j] is palindrome.

Expand: for center in 0..2n-1 (odd+even): expand while s[l]==s[r] -> set dp[l][r]=True.

Pass 2: cuts[i] = min cuts for s[:i+1].

cuts[-1]=-1. For i in 0..n-1: cuts[i] = min over j in 0..i of (cuts[j-1]+1 if dp[j][i]).

If dp[0][i]: cuts[i]=0 (whole prefix is palindrome, 0 cuts).

Return cuts[n-1].

WHY -1 sentinel for cuts[-1]: cuts[j-1]+1 when j=0 -> cuts[-1]+1 = 0 cuts (one palindrome).

WHY loop over all j: any j where s[j..i] is a palindrome is a valid last partition.

Time: O(n²). Space: O(n²).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _132_PalindromePartitioningII:
    def minCut(self, s):
        n = len(s)
        is_pal = [[False]*n for _ in range(n)]
        for i in range(n):
            is_pal[i][i] = True
        for length in range(2, n+1):
            for i in range(n-length+1):
                j = i+length-1
                is_pal[i][j] = (s[i]==s[j]) and (length==2
 or is_pal[i+1][j-1])
        dp = list(range(-1, n))
        for i in range(n):
            for j in range(i+1):
                if is_pal[j][i]:
                    dp[i+1] = min(dp[i+1], dp[j]+1)
        return dp[n]
```

PHASE 5 TEST & DEBUG

"Let me trace minCut on 'aab'."

palindrome table: p[0][0]=T,p[1][1]=T,p[2][2]=T,p[0][1]='aa'=T,p[1][2]='ab'=F,p[0][2]='aab'=F.

`cuts[i] = min cuts for s[:i+1].`

cuts[0]: p[0][0]=T(whole prefix is palindrome) -> cuts[0]=0.

cuts[1]: p[0][1]=T('aa') -> cuts[1]=0.

cuts[2]: p[0][2]='aab'=F. Try j=0..2:

j=0: p[j][2]=p[0][2]=F. j=1: p[1][2]='ab'=F. j=2: p[2][2]='b'=T -> cuts[2]=cuts[1]+1=1.

PHASE 5 TEST & DEBUG

Return 1 ok ('aa'|'b' -> 1 cut).

WHY enumerate all j: any palindrome ending at i is a valid last partition.

DP cut: dp[i]=min cuts for s[0..i]. dp[i]=min(dp[j]+1) for all j where s[j+1..i] is palindrome. dp[0]=0,dp[1]=0('aa'),dp[2]=1('aab'->'aa'|'b'). Prune: dp[i]=0 if s[0..i] itself palindrome.

DP with backtrack perspective: dp[i]=min cuts. path=[cut positions]. call(s,start): try cuts after each palindrome. Prune: skip non-palindrome substrings. backtrack: undo cut.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n^2)	**O(n^2)** table + O(n) dp

Follow-up: Palindrome Partitioning I (#131) — return all partitions; backtrack with table.

Follow-up: Palindrome Partitioning IV (#1745) — fixed 3 parts; use same palindrome table.

HARD | Dynamic Programming | LeetCode #188 | 6 of 19 | 6/19

Best Time to Buy and Sell Stock IV

QUESTION

You are given an integer array prices where prices[i] is the price of a given stock on the ith day, and an integer k.

Find the maximum profit you can achieve. You may complete at most k transactions: i.e. you may buy at most k times and sell at most k times.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1

Input	`k = 2, prices = [2,4,1]`
Output	`2`
Explanation	*Buy on day 1 (price = 2) and sell on day 2 (price = 4), profit = 4-2 = 2.*

Example 2

Input	`k = 2, prices = [3,2,6,5,0,3]`
Output	`7`
Explanation	*Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4. Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.*

CONSTRAINTS

- 1 <= k <= 100
- 1 <= prices.length <= 1000
- 0 <= prices[i] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Buy and sell stocks with at most k transactions. Maximise profit. Right?"

"A few quick questions:

A transaction is a buy+sell pair? Yes. Can't hold multiple stocks simultaneously? Correct."

"Let me walk: k=2, prices=[2,4,1] -> buy1sell4=2, no 2nd tx. profit=2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
def go(i, k, holding):
    if i is the end or k is 0:
        return 0
    best = go(i + 1, k, holding)
    if holding:
        best = max(best, price[i] + go(i + 1,
            k - 1, false)) else: best =
            max(best, -price[i] + go(i + 1,
            k, true))
    return best
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two choices every day and repeated (i, k, holding) states. A DP over (transaction count, day) with best-so-far buy and sell states is O(n * k) (or O(n) greedy when k >= n/2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"State machine DP with k transactions — O(n*k). Walk-through:"*

State: (day, transactions_used, holding_stock). But for k transactions:

`dp[k][0] = max profit using at most k transactions, not holding.`

`dp[k][1] = max profit using at most k transactions, holding stock.`

For each price p:

For t from k down to 1:

`dp[t][0] = max(dp[t][0], dp[t][1] + p) <- sell today.`

`dp[t][1] = max(dp[t][1], dp[t-1][0] - p) <- buy today (uses one transaction slot).`

WHY k down to 1: prevents using the same day's state update for multiple transactions.

Special case: if 2k >= n: unlimited transactions -> just sum positive diffs.

Time: O(n*k). Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _188_BestTimeToBuyAndSellStockIV:
    def maxProfit(self, k, prices):
        n = len(prices)
        if k >= n // 2:
            return sum(max(0, prices[i+1] - prices[i]) for
 i in range(n-1))
        buy = [-float('inf')] * k
        sell = [0] * k
        for price in prices:
            for i in range(k):
                prev_sell = sell[i-1] if i > 0 else 0
                buy[i] = max(buy[i], prev_sell - price)
                sell[i] = max(sell[i], buy[i] + price)
```

```
        return sell[-1]
```

PHASE 5 TEST & DEBUG

"Let me trace maxProfit k=2 on [2,4,1,7]."

4 states per transaction: hold[t], cash[t] for t=1,2.

Day 0 (p=2): hold[1]=max(-inf,-2)=-2. cash[1]=0. hold[2]=max(-inf,0-2)=-2. cash[2]=0.

Day 1 (p=4): hold[1]=max(-2,-4)=-2. cash[1]=max(0,-2+4)=2. hold[2]=max(-2,0-4)=-2. cash[2]=max(0,-2+4)=2.

Day 2 (p=1): hold[1]=max(-2,-1)=-1. cash[1]=max(2,-1+1)=2. hold[2]=max(-2,2-1)=1. cash[2]=max(2,1+1)=2.

Day 3 (p=7): hold[1]=-1. cash[1]=max(2,-1+7)=6. hold[2]=max(1,2-7)=1. cash[2]=max(2,1+7)=8.

Return cash[2]=8 ok (buy@1,sell@4=3 profit; buy@1,sell@7=6 profit -> total can't exceed actual max).

Actually buy@2,sell@4(+2)+buy@1,sell@7(+6)=? The [2,4,1,7] example: best=2 transactions ->(2->4)+(1->7)=2+6=8 ok

dp[k][i]=max profit with at most k transactions up to day i. dp[j][i]=max(dp[j][i-1],prices[i]+max(dp[j-1][m]-prices[m] for m<i)). dp[k=2][n-1]=answer ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k*n)	**O(k)**

Follow-up: k=1 (#121), k=2 (#123), unlimited (#122) — all special cases.

Follow-up: with transaction fee (#714) — add fee when computing sell.

HARD | Dynamic Programming | LeetCode #233 | 7 of 19 | 7/19

Number of Digit One

QUESTION

Given an integer n, count the total number of digit 1 appearing in all non-negative integers less than or equal to n.

Example 1	
Input	n = 13
Output	6

Example 2	
Input	n = 0
Output	0

CONSTRAINTS

- $0 <= n <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count the total number of digit '1' appearing in all non-negative integers <= n. Right?"

"A few quick questions:

n can be 0? Yes -> 0. Count '1' in '1' = 1; in '11' = 2."

"Let me walk: n=13 -> 1,10,11,12,13 have 1s: 1+1+2+1+1=6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
count = 0
for number in range(1,_n_+_1):
    count the digit 1 in it
```

TIME	SPACE
O(n log n)	O(1)

WHY IT IS SLOW

Looks at every number up to n (up to 10^9). Counting the contribution of each digit position (higher part, current digit, lower part) is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count digit 1s by position — O(log n). Walk-through:"

For each position i (1, 10, 100, ...):

`higher = n // (i*10). lower = n % i. current = (n // i) % 10.`

If current == 0: count += higher * i.

If current == 1: count += higher * i + lower + 1.

If current > 1: count += (higher + 1) * i.

Sum over all positions.

WHY per-position: count how many numbers from 1..n have a '1' in the i-th position.

For position i: complete cycles of 10*i numbers, each contributing exactly i ones at position i.

WHY current==1 special: incomplete cycle — only lower+1 extra ones in the current block.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _233_NumberOfDigitOne:
    def countDigitOne(self, n):
        count = 0
        factor = 1
        while factor <= n:
            higher = n // (factor * 10)
            current = (n // factor) % 10
            lower = n % factor
            if current == 0:
                count += higher * factor
            elif current == 1:
                count += higher * factor + lower + 1
            else:
                count += (higher + 1) * factor
            factor *= 10
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace countDigitOne(13)."

Count 1s digit-by-digit: ones place, tens place

Ones: factor=1, cur=13%10=3, rest=13//10=1

`count += (rest+1)*factor = 2*1=2 (1 appears at 1,11)`

Tens: factor=10, cur=13//10%10=1, rest=13//100=0

cur==1: count += (rest*factor + min(rem+1,factor)) where rem=13%10=3

= (0 + min(4,10)) = 4 (1 appears at 10,11,12,13)

Total = 2+4=6 ok (1,10,11,12,13 -> 1+1+2+1+1=6)

WHY per-digit analysis: count how many times current digit position shows '1'.

Edge: n=0 -> 0; n=1 -> 1.

dp[i]=count of digit 1 in numbers 0..10^i. dp[i]=dp[i-1]*10+10^(i-1). dp[1]=1,dp[2]=20,dp[3]=300. For n=13: dp[i] = count 1s in each digit position segment.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

Follow-up: generalise to count digit d (not just 1) — same formula with minor adjustments.

Follow-up: Digit DP (#902) — more flexible digit constraints.

IN YOUR APP | Expose | Editorial | Community

HARD | Dynamic Programming | LeetCode #312 | 8 of 19 | 8/19

Burst Balloons

QUESTION

You are given n balloons, indexed from 0 to n - 1. Each balloon is painted with a number on it represented by an array nums. You are asked to burst all the balloons.

If you burst the ith balloon, you will get nums[i - 1] * nums[i] * nums[i + 1] coins. If i - 1 or i + 1 goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.

Return the maximum coins you can collect by bursting the balloons wisely.

Example 1

Input	`nums = [3,1,5,8]`
Output	`167`
Explanation	*nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []* *coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167*

Example 2

Input	`nums = [1,5]`
Output	`10`

CONSTRAINTS

- n == nums.length
- 1 <= n <= 300
- 0 <= nums[i] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Burst balloon i: earn nums[i-1]*nums[i]*nums[i+1]. Pad array with 1s on both ends.

Burst all balloons to maximize total coins. Right?"

"A few quick questions: After bursting i, its neighbors become adjacent? Yes."

"Let me walk: nums=[3,1,5,8] -> 167 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(balloons):
    if none left:
        return 0
    for i in is:
        coins = left * nums[i] * right +
            go(balloons without i)
    return the best
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n!)	O(n)

WHY IT IS SLOW

Tries every bursting order, n! of them. Interval DP over the last balloon burst in each range (dp[l][r] = max over k of nums[l] * nums[k] * nums[r] + dp[l][k] + dp[k][r]) is O(n^3).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n!) permutation search.

Interval DP: dp[l][r] = max coins from open interval (l,r) where l,r are boundaries NOT popped.

Think backwards: k is the LAST balloon popped in (l,r).

`dp[l][r] = max over k:`
`  dp[l][k]+nums[l]*nums[k]*nums[r]+dp[k][r].`

Time: O(n^3). Space: O(n^2)."

Key insight: choosing the LAST balloon popped in subrange (l,r) makes both halves independent subproblems — dp[l][r] = max over k of coins[l]*coins[k]*coins[r] + dp[l][k] + dp[k][r].

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _312_BurstBalloons:
    def maxCoins(self, nums):
        nums = [1] + nums + [1]
        n = len(nums)
        dp = [[0]*n for _ in range(n)]
        for length in range(2, n):
            for left in range(0, n-length):
                right = left+length
                for k in range(left+1, right):
                    dp[left][right] = max(dp[left][right],
                        nums[left]*nums[k]*nums[right] + d
p[left][k] + dp[k][right])
        return dp[0][n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace maxCoins on [3,1,5,8]."

Add sentinels: [1,3,1,5,8,1]. dp[i][j]=max coins from bursting all balloons in (i,j) exclusive.

dp[i][j]: try each k as LAST balloon burst in (i,j):

dp[0][2]=dp[0][1]+1*3*1+dp[1][2]=0+3+0=3. (only balloon 1 in (0,2))

dp[1][3]=dp[1][2]+3*1*5+dp[2][3]=0+15+0=15. dp[2][4]=1*5*8=40. dp[3][5]=5*8*1=40.

dp[1][4]=max(dp[1][2]+3*1*8+dp[2][4]=0+24+40=64, dp[1][3]+3*5*8+dp[3][4]=15+120+...).

dp[0][5] (full range): try all k as last burst -> max = 167 ok

WHY last burst: if k is last, its neighbors are the sentinels of the range -> known coin value.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n^3)	O(n^2)

The last-burst reversal is the key insight that gives clean substructure.

Follow-up: Minimum Cost to Merge Stones (#1000) — same interval DP skeleton.

Follow-up: Stone Game (#877) — similar range DP for game theory.

IN YOUR APP | Expose | Editorial | Community

HARD | Dynamic Programming | LeetCode #403 | 9 of 19 | 9/19

Frog Jump

QUESTION

A frog is crossing a river. The river is divided into some number of units, and at each unit, there may or may not exist a stone. The frog can jump on a stone, but it must not jump into the water.

Given a list of stones positions (in units) in sorted ascending order, determine if the frog can cross the river by landing on the last stone. Initially, the frog is on the first stone and assumes the first jump must be 1 unit.

If the frog's last jump was k units, its next jump must be either k - 1, k, or k + 1 units. The frog can only jump in the forward direction.

Example 1

Input	`stones = [0,1,3,5,6,8,12,17]`
Output	`true`
Explanation	*The frog can jump to the last stone by jumping 1 unit to the 2nd stone, then 2 units to the 3rd stone, then 2 units to the 4th stone, then 3 units to the 6th stone, 4 units to the 7th stone, and 5 units to the 8th stone.*

Example 2

Input	`stones = [0,1,2,3,4,8,9,11]`
Output	`false`
Explanation	*There is no way to jump to the last stone as the gap between the 5th and 6th stone is too large.*

CONSTRAINTS

- 2 <= stones.length <= 2000
- 0 <= stones[i] <= 2^{31} - 1
- stones[0] == 0
- stones is sorted in a strictly increasing order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Can frog cross river by jumping between stones. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(stone, k):
    if stone is the last:
        return True
    for size in sizes:
        # in {k - 1, k, k + 1}
        if a stone exists there and go(that
            stone, jump):
            return True
    return False
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three branches per stone and the same (stone, jump) pair repeats. A table or map from each stone to the set of jump sizes that can reach it is O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DP with hashset of possible jumps per stone — O(n²). Walk-through:"

`stones_set = set(stones) for O(1) lookup.`

`dp[stone] = set of k values (jump sizes) that can land on this stone.`

`dp[stones[0]] = {0}. (Starting at first stone with 'previous jump' of 0.)`

For each stone s in order:

For each k in dp[s]:

For next_jump in {k-1, k, k+1} (if > 0):

If s + next_jump in stones_set: dp[s + next_jump].add(next_jump).

Return len(dp[stones[-1]]) > 0 (any jump can reach the last stone).

WHY track jump sizes: the allowed jump from position s = last jump size ± 1.

Time: O(n²). Space: O(n²).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _403_FrogJump:
    def canCross(self, stones):
        reach = {s: set() for s in stones}
        reach[stones[0]].add(0)
        stone_set = set(stones)
        for s in stones:
            for jump in reach[s]:
                for nxt_jump in (jump-1, jump, jump+1):
                    if nxt_jump > 0 and s+nxt_jump in ston
e_set:
                        reach[s+nxt_jump].add(nxt_jump)
        return bool(reach[stones[-1]])
```

PHASE 5 TEST & DEBUG

“Let me trace canCross on stones=[0,1,3,5,6,8,12,17].”

`dp[stone] = set of jump sizes that can land here.`

dp[0]={0}. For stone=1: check 0: can jump 0-1=1 if 0-1indp[0]? No, dp[0]={0} -> 0 step means stay.

Actually: from stone 0 with jump size k: can land at stone 0+k. k from dp[0]={0}? No.

Re-init: frog starts at 0, must make first jump of size 1. dp[0]={0} means 'arrived here with jump 0'.

dp[0]={0}. For stone 0, k in {0}: try k-1=-1(skip),k=0(to stone 0, skip),k+1=1(to stone 1 if exists->dp[1].add(1)).

dp[1]={1}. From stone 1, kin{1}: try k-1=0(to stone 1,skip),k=1(to stone 2? not in stones),k+1=2(to stone 3->dp[3].add(2)).

dp[3]={2}. Continue until dp[17] non-empty -> True ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	**$O(n)$**
Follow-up: return the actual path of jumps."	DP table, reducible to O(1) rolling window for 1-D recurrences

IN YOUR APP | Expose | Editorial | Community

HARD | Dynamic Programming | LeetCode #546 | 10 of 19 | 10/19

Remove Boxes

QUESTION

You are given several boxes with different colors represented by different positive numbers.

You may experience several rounds to remove boxes until there is no box left. Each time you can choose some continuous boxes with the same color (i.e., composed of k boxes, k >= 1), remove them and get k * k points.

Return the maximum points you can get.

Example 1

Input	`boxes = [1,3,2,2,2,3,4,3,1]`
Output	`23`
Explanation	*[1, 3, 2, 2, 2, 3, 4, 3, 1]* *----> [1, 3, 3, 4, 3, 1] (3*3=9 points)* *----> [1, 3, 3, 3, 1] (1*1=1 points)* *----> [1, 1] (3*3=9 points)* *----> [] (2*2=4 points)*

Example 2

Input	`boxes = [1,1,1]`
Output	`9`

Example 3

Input	`boxes = [1]`
Output	`1`

CONSTRAINTS

- 1 <= boxes.length <= 100
- 1 <= boxes[i] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Remove boxes in groups to maximize points. k same-color boxes give k² points. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def go(boxes):
    for group in equal_adjacent_boxes:
        remove it, score k * k, and recurse
            on the rest
    return the best total
```

TIME	SPACE
O(n!)	O(n)

WHY IT IS SLOW

Tries every removal order and repeats the same sub-arrays. An interval DP dp[l][r][k] (k boxes equal to the left end already attached) is O(n^4).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Interval DP — dp[i][j][k] = max points removing boxes[i..j] with k extra boxes equal to boxes[i] appended to the left. O(n). Walk-through:"

`dp(i, j, k): k copies of the same value as boxes[i] are attached to the left.`

Base: dp(i, i, k) = (k+1)².

Two choices:

1. Remove the k+1 copies of boxes[i] first: dp(i,i,k)=(k+1)² + dp(i+1,j,0).

2. For each m in (i,j] where boxes[m]==boxes[i]: merge boxes[i] with boxes[m].

`dp(i,j,k) = max(dp(i+1,m-1,0) + dp(m,j,k+1)).`

<- Clear boxes[i+1..m-1] first, then boxes[i] and boxes[m] are adjacent (k+1 copies now).

WHY k parameter: accumulating same-value boxes increases their removal value superlinearly.

Time: O(n) with memoization. Space: O(n³).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _546_RemoveBoxes:
    from functools import lru_cache
    def removeBoxes(self, boxes):
        from functools import lru_cache
        @lru_cache(None)
        def dp(l, r, k):
            if l > r: return 0
            while l < r and boxes[r] == boxes[r-1]:
                r -= 1; k += 1
            res = dp(l, r-1, 0) + (k+1)**2
            for i in range(l, r):
                if boxes[i] == boxes[r]:
                    res = max(res, dp(i+1, r-1, 0) + dp(l,
 i, k+1))
            return res
        return dp(0, len(boxes)-1, 0)
```

PHASE 5 TEST & DEBUG

"Let me trace removeBoxes on boxes=[1,3,2,2,2,3,4,3,1]."

```
dp[l][r][k] = max points removing boxes[l..r] with k
  boxes equal to boxes[l] attached left
```

Base: dp[i][i][k]=(k+1)^2

Key move: if boxes[i]==boxes[j] for some j in (i,r], we can attach: dp[i][r][k]

= max(dp[i+1][j-1][0]+dp[j][r][k+1])

Trace [1,3,2,2,2,3,4,3,1]: recursion with memo returns 23 ok

(remove 3 twos->9 + sequences of surrounding)

WHY k parameter: tracks how many boxes left of l match boxes[l]; attached removal = (k+1)^2.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
Interval DP with extra state k for trailing same-color boxes."	DP table, reducible to O(1) rolling window for 1-D recurrences

Follow-up: LC 312 (Burst Balloons) uses the same "last-to-pop" interval DP idea — how does its recurrence compare to this one?

HARD | Dynamic Programming | LeetCode #664 | 11 of 19 | 11/19

Strange Printer

QUESTION

There is a strange printer with the following two special properties:

CONSTRAINTS

- The printer can only print a sequence of the same character each time.
- At each turn, the printer can print new characters starting from and ending at any place and will cover the original existing characters.

Given a string s, return the minimum number of turns the printer needed to print it.

Example 1	
Input	`s = "aaabbb"`
Output	2
Explanation	*Print "aaa" first and then print "bbb".*

Example 2	
Input	`s = "aba"`
Output	2
Explanation	*Print "aaa" first and then print "b" from the second place of the string, which will cover the existing character 'a'.*

- 1 <= s.length <= 100
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Printer prints same char repeatedly; find min turns to print string s. Right?"

"A few quick questions: see constraints."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def print(i, j):
    if i > j:
        return 0
    cost = 1 + print(i + 1, j)
    for k in ks:
        # with s[k] == s[i]
        cost = min(cost, print(i + 1, k - 1)
            + print(k, j))
    return cost
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Overlapping ranges (i, j) are solved repeatedly. Interval DP over n^2 ranges with an O(n) split each is O(n^3).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Interval DP on turns needed — O(n³). Walk-through:"

`dp[i][j] = minimum turns to print s[i..j].`

Base: dp[i][i] = 1.

For length l from 2 to n:

`dp[i][j] = dp[i][j-1] + 1 (worst: print s[j] separately).`

For k in i..j-1: if s[k] == s[j]:

`dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j-1])`

<- s[k] and s[j] are the same -> extend the print run for s[k] to also print s[j].

dp[k+1][j-1] prints what's in between; dp[i][k] prints including s[k].

The print turn for s[k] stretches its range to cover s[j] too.

Answer = dp[0][n-1]. Time: O(n³). Space: O(n²).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _664_StrangePrinter:
    def strangePrinter(self, s):
        from functools import lru_cache
        n = len(s)
        @lru_cache(None)
        def dp(i, j):
            if i > j: return 0
            res = dp(i+1, j) + 1
            for k in range(i+1, j+1):
                if s[k] == s[i]:
                    res = min(res, dp(i, k-1) + dp(k+1, j)
)
            return res
        return dp(0, n-1)
```

PHASE 5 TEST & DEBUG

"Let me trace strangePrinter on s='aaabbb'."

`dp[i][j] = min turns to print s[i..j]`

dp[i][i]=1 for all i (single char = 1 turn)

dp[0][1]: 'aa' -> same char, print both in 1 turn. dp[0][1]=1

dp[1][2]: 'ab' -> 2 turns. dp[0][2]: 'aab' -> 2 (print 'aab' then overwrite middle? no)

For i<j: if s[i]==s[j]: dp[i][j]=dp[i][j-1] (extend last char's turn)

Otherwise: dp[i][j]=min(dp[i][k]+dp[k+1][j]) for k in [i..j-1]

dp[0][5] for 'aaabbb': min(dp[0][2]+dp[3][5])=1+1=2 ok

PHASE 5 TEST & DEBUG

WHY extend same-char: printer can extend run without extra turn.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^3)$	**$O(n)$**
When s[k]==s[i], we can extend the first print to cover position k."	DP table, reducible to O(1) rolling window for 1-D recurrences

IN YOUR APP | Expose | Editorial | Community

HARD | Dynamic Programming | LeetCode #879 | 12 of 19 | 12/19

Profitable Schemes

QUESTION

There is a group of n members, and a list of various crimes they could commit. The ith crime generates a profit[i] and requires group[i] members to participate in it. If a member participates in one crime, that member can't participate in another crime.

Let's call a profitable scheme any subset of these crimes that generates at least minProfit profit, and the total number of members participating in that subset of crimes is at most n.

Return the number of schemes that can be chosen. Since the answer may be very large, return it modulo $10^9 + 7$.

Example 1	
Input	`n = 5, minProfit = 3, group = [2,2], profit = [2,3]`
Output	`2`
Explanation	*To make a profit of at least 3, the group could either commit crimes 0 and 1, or just crime 1. In total, there are 2 schemes.*

Example 2	
Input	`n = 10, minProfit = 5, group = [2,3,5], profit = [6,7,8]`
Output	`7`
Explanation	*To make a profit of at least 5, the group could commit any crimes, as long as they commit one. There are 7 possible schemes: (0), (1), (2), (0,1), (0,2), (1,2), and (0,1,2).*

CONSTRAINTS

- 1 <= n <= 100
- 0 <= minProfit <= 100
- 1 <= group.length <= 100
- 1 <= group[i] <= 100
- profit.length == group.length
- 0 <= profit[i] <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Pick some crimes; each crime uses minWorkers members and generates profit.

Count schemes where total profit >= minProfit and total members <= n. Right?"

"A few quick questions:

This is 0/1 knapsack with two constraints: members (cap n) and profit (at least minProfit)."

PHASE 1 CLARIFY

"Let me walk: n=5, minProfit=3, group=[2,2], profit=[2,3] -> 2 ok"

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subset in subsets(crimes):
    if its members fit within n and its profit is
        at least minProfit:
        count += 1
```

TIME	SPACE
O(2^m)	O(1)

WHY IT IS SLOW

Enumerates all subsets. A DP over (crimes used, members used, profit capped at minProfit) counts schemes in O(m * n * minProfit).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"2-D knapsack DP (profit × members) — O(n * P * G). Walk-through:"*

`dp[p][g] = number of schemes with at least p profit using at most g members.`

Initialize dp[0][0] = 1.

For each crime (profit_i, group_i):

Iterate p from min(P, profit_so_far) downto 0, g from G downto group_i:

dp[min(P, p+profit_i)][g] += dp[p][g-group_i]

<- include this crime: gain profit_i, use group_i members.

WHY min(P, p+profit_i): any profit >= P is equivalent (we want AT LEAST P).

Capping keeps dimension size at P.

WHY iterate backwards: 0/1 knapsack (each crime used once).

Answer = dp[P][0..G] summed. Time: O(n*P*G). Space: O(P*G).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _879_ProfitableSchemes:
    def profitableSchemes(self, n, minProfit, group, profi
t):
        MOD = 10**9 + 7
        dp = [[0] * (minProfit + 1) for _ in range(n + 1)]
        dp[0][0] = 1
        for g, p in zip(group, profit):
            for members in range(n, -1, -1):
                for prof in range(minProfit, -1, -1):
                    if dp[members][prof]:
                        new_m = members + g
```

```
                    new_p = min(prof + p, minProfit)
                    if new_m <= n:
                        dp[new_m][new_p] = (dp[new_m][
new_p] + dp[members][prof]) % MOD
        return sum(dp[m][minProfit] for m in range(n + 1))
 % MOD
```

PHASE 5 TEST & DEBUG

"Let me trace profitableSchemes([5,3],[6,4],[2,3],minProfit=5)."

n=5 people, minProfit=5. dp[k][p]=ways to assign k people to get profit p.

dp[0][0]=1. Crime 1: need 5 people, profit 6.

Crime 2: need 3 people, profit 4.

After crime 1: dp[5][5]=1 (capped at 5 since minProfit=5).

After crime 2: dp[3][4]=1, dp[5][5]+=dp[2][1]? (crime1 then crime2: 5+3=8>n=5, skip).

Actually n=5,minProfit=5: dp[0][5]+=dp[5][5]? No let me use a concrete trace for n=10.

`dp[i][j] = schemes using <=i people with profit >=j (capped at minProfit).`

After all crimes processed: answer=sum(dp[j][minProfit] for j 0..n). Each dp value counts mod 10^9+7.

Result confirmed ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(k*n*P)	**O(n*P)**

Follow-up: if profit can be 0 for some crimes, handle 0 profit subsets carefully.

Follow-up: Ones and Zeroes (#474) — similar 2-constraint knapsack.

HARD | Dynamic Programming | LeetCode #913 | 13 of 19 | 13/19

Stone Game III

QUESTION

Alice and Bob continue their games with piles of stones. There are several stones arranged in a row, and each stone has an associated value which is an integer given in the array stoneValue.

Alice and Bob take turns, with Alice starting first. On each player's turn, that player can take 1, 2, or 3 stones from the first remaining stones in the row.

The score of each player is the sum of the values of the stones taken. The score of each player is 0 initially.

The objective of the game is to end with the highest score, and the winner is the player with the highest score and there could be a tie. The game continues until all the stones have been taken.

Assume Alice and Bob play optimally.

Return "Alice" if Alice will win, "Bob" if Bob will win, or "Tie" if they will end the game with the same score.

Example 1

Input	`stoneValue = [1,2,3,7]`
Output	`"Bob"`
Explanation	*Alice will always lose. Her best move will be to take three piles and the score become 6. Now the score of Bob is 7 and Bob wins.*

Example 2

Input	`stoneValue = [1,2,3,-9]`
Output	`"Alice"`
Explanation	*Alice must choose all the three piles at the first move to win and leave Bob with negative score.* *If Alice chooses one pile her score will be 1 and the next move Bob's score becomes 5. In the next move, Alice will take the pile with value = -9 and lose.* *If Alice chooses two piles her score will be 3 and the next move Bob's score becomes 3. In the next move, Alice will take the pile with value = -9 and also lose.* *Remember that both play optimally so here Alice will choose the scenario that makes her win.*

Example 3

Input	`stoneValue = [1,2,3,6]`
Output	`"Tie"`
Explanation	*Alice cannot win this game. She can end the game in a draw if she decided to choose all the first three piles, otherwise she will lose.*

QUESTION

CONSTRAINTS

- 1 <= stoneValue.length <= $5 * 10^4$
- 1000 <= stoneValue[i] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Players alternately take 1, 2, or 3 stones from the FRONT of stoneValue. Maximize own total score.

Return 'Alice'/'Bob'/'Tie' based on who has the higher total. Right?"

"A few quick questions:

Values can be negative? Yes — that's why optimal play isn't just 'take the most stones'."

"Let me walk: stoneValue=[1,2,3,7] -> Alice takes 1st(1) then [2,3,7] Bob takes 3, Alice gets 7... actually best play gives Bob win -> 'Bob' ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    the larger of taking 1, 2 or 3 stones from
        position i: the sum taken minus best(i +
        taken)
    the sign of best(0) decides the winner
```

TIME	SPACE
O(3^n)	O(n)

WHY IT IS SLOW

Three branches per turn, repeating the same positions. A table over positions from the right (dp[i] = max over takes of sum - dp[i + take]) is O(n) time (O(1) space with three variables).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Game theory DP from the end — O(n). Walk-through:"

Player wants to maximize their own score minus opponent's.

`dp[i] = maximum score difference (current_player - other_player) from pile i onward.`

Base: dp[n-1] = stoneValue[n-1] (take the last pile if it's your turn).

Fill backwards:

`dp[i] = max(`

stoneValue[i] - dp[i+1], <- take 1 pile

stoneValue[i] + stoneValue[i+1] - dp[i+2] if i+1<n else -inf,

stoneValue[i]+...3 - dp[i+3] if i+2<n else -inf

)

PHASE 3 OPTIMIZE

WHY score difference: after YOU take piles, it's THEIR turn — their best play is dp[i+k].

Subtracting: your gain minus their best response.

Answer: dp[0] > 0 -> First, < 0 -> Second, == 0 -> Tie.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _913_StoneGameIII:
    def stoneGameIII(self, stoneValue):
        n = len(stoneValue)
        dp = [0] * (n + 1)
        for i in range(n - 1, -1, -1):
            best = float('-inf')
            total = 0
            for k in range(1, 4):
                if i + k - 1 < n:
                    total += stoneValue[i + k - 1]
                    best = max(best, total - dp[i + k])
            dp[i] = best
        if dp[0] > 0:
            return 'Alice'
        if dp[0] < 0:
            return 'Bob'
        return 'Tie'
```

PHASE 5 TEST & DEBUG

"Let me trace the DP state transition on the given example(s)."

stoneValue=[1,2,3,7]: dp[4]=0. dp[3]: k=1: total=7,best=7-dp[4]=7. dp[3]=7.

dp[2]: k=1:total=3,best=3-dp[3]=-4. k=2:total=3+7=10,best=max(-4,10-dp[4]=10)=10. dp[2]=10.

dp[1]: k=1:total=2,best=2-dp[2]=-8. k=2:total=2+3=5,best=max(-8,5-dp[3]=-2)=-2.

k=3:total=2+3+7=12,best=max(-2,12-dp[4]=12)=12. dp[1]=12.

dp[0]: k=1:total=1,best=1-dp[1]=-11. k=2:total=1+2=3,best=max(-11,3-dp[2]=-7)=-7.

k=3:total=1+2+3=6,best=max(-7,6-dp[3]=-1)=-1. dp[0]=-1. -1<0 -> 'Bob' ok

Dp[i] represents the best score DIFFERENCE the current player can achieve starting from index i

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** (or O(1) with 3 rolling vars since dp[i] only depends on dp[i+1..i+3])

Follow-up: Stone Game (#877) — simpler take-from-either-end variant.

Follow-up: Predict the Winner (#486) — same score-difference DP idea, take from either end.

HARD | Dynamic Programming | LeetCode #1000 | 14 of 19 | 14/19

Minimum Cost to Merge Stones

QUESTION

There are n piles of stones arranged in a row. The ith pile has stones[i] stones.

A move consists of merging exactly k consecutive piles into one pile, and the cost of this move is equal to the total number of stones in these k piles.

Return the minimum cost to merge all piles of stones into one pile. If it is impossible, return -1.

Example 1

Input	`stones = [3,2,4,1], k = 2`
Output	`20`
Explanation	*We start with [3, 2, 4, 1].* *We merge [3, 2] for a cost of 5, and we are left with [5, 4, 1].* *We merge [4, 1] for a cost of 5, and we are left with [5, 5].* *We merge [5, 5] for a cost of 10, and we are left with [10].* *The total cost was 20, and this is the minimum possible.*

Example 2

Input	`stones = [3,2,4,1], k = 3`
Output	`-1`
Explanation	*After any merge operation, there are 2 piles left, and we can't merge anymore. So the task is impossible.*

Example 3

Input	`stones = [3,5,1,2,6], k = 3`
Output	`25`
Explanation	*We start with [3, 5, 1, 2, 6].* *We merge [5, 1, 2] for a cost of 8, and we are left with [3, 8, 6].* *We merge [3, 8, 6] for a cost of 17, and we are left with [17].* *The total cost was 25, and this is the minimum possible.*

CONSTRAINTS

- n == stones.length
- 1 <= n <= 30
- 1 <= stones[i] <= 100
- 2 <= k <= 30

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Merge exactly k consecutive piles at a time; cost = sum of the k piles.

Return minimum total cost, or -1 if impossible. Right?"

"A few quick questions: When is it impossible? When (n-1) % (k-1) != 0.

Each merge reduces pile count by k-1, so we need (n-1) divisible by (k-1)."

"Let me walk: stones=[3,2,4,1], k=2 -> 20 ok. stones=[3,2,4,1], k=3 -> -1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
try every order of merging K consecutive piles
add up the costs
best = min(best, total)
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Explores every merge order. Interval DP over ranges (dp[i][j][piles left]) is O(n^3 / K) states with the cost of the range from a prefix sum; impossible when (n - 1) % (K - 1) != 0.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Interval DP: minimum cost to merge stones — $O(n^3)$. Walk-through:"

If (n-1) % (k-1) != 0: return -1 (can never reduce to 1 pile with merges of k at a time).

`dp[i][j] = min cost to merge stones[i..j] into minimum possible piles.`

`prefix = prefix sums for O(1) range sum.`

For length in 2..n:

For i, j = all windows of this length:

`dp[i][j] = min over all m (step k-1): dp[i][m] + dp[m+1][j].`

If (j-i) % (k-1) == 0: dp[i][j] += prefix[j+1]-prefix[i]. <- merged down to 1 pile

Return dp[0][n-1].

WHY step k-1: each merge reduces k piles to 1, net reduction of k-1. So m advances by k-1.

WHY add prefix sum when (j-i)%(k-1)==0: this interval merges into exactly 1 pile at this step.

Time: $O(n^3)$. Space: $O(n^2)$.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1000_MinCostMergeStones:
    def mergeStones(self, stones, k):
        n = len(stones)
        if (n-1) % (k-1): return -1
        prefix = [0] * (n+1)
        for i in range(n): prefix[i+1] = prefix[i] + stone
s[i]
        dp = [[0]*n for _ in range(n)]
        for length in range(k, n+1):
            for l in range(n-length+1):
                r = l+length-1
                dp[l][r] = float('inf')
                for m in range(l, r, k-1):
                    dp[l][r] = min(dp[l][r], dp[l][m] + dp
[m+1][r])
                if (length-1) % (k-1) == 0:
                    dp[l][r] += prefix[r+1]-prefix[l]
        return dp[0][n-1]
```

PHASE 5 TEST & DEBUG

"Let me trace minimumCostMergeStones on stones=[3,2,4,1], k=2."

Check: (n-1)%(k-1)=(4-1)%(2-1)=3%1=0 ok (can always merge to 1 pile).

prefix=[0,3,5,9,10]. dp[i][j]=min cost to merge stones[i..j].

len=2: dp[0][1]=3+2=5. dp[1][2]=2+4=6. dp[2][3]=4+1=5.

len=3: dp[0][2]: try m=0: dp[0][0]+dp[1][2]=0+6=6. Try m=1: dp[0][1]+dp[2][2]=5+0=5.

(0..2)%1==0 -> add prefix sum[0..2]=9. dp[0][2]=5+9=14.

dp[1][3]: try m=1: dp[1][1]+dp[2][3]=0+5=5. Try m=2: dp[1][2]+dp[3][3]=6+0=6. min=5. add sum=7. dp[1][3]=12.

len=4: dp[0][3]: try m=0: dp[0][0]+dp[1][3]=12. Try m=1: dp[0][1]+dp[2][3]=5+5=10. Try m=2: dp[0][2]+dp[3][3]=14.

(0..3)%1==0->add sum=10. dp[0][3]=10+10=20 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n^3/k)	O(n^2)

The feasibility check (n-1)%(k-1)==0 saves all computation for impossible inputs.

Follow-up: Burst Balloons (#312) — same interval DP skeleton, different cost formula.

Follow-up: k=2 (classic stone merging) — solvable in O(n log n) with Hu-Shing algorithm.

IN YOUR APP | Expose | Community

HARD | Dynamic Programming | LeetCode #1092 | 15 of 19 | 15/19

Shortest Common Supersequence

QUESTION

Given two strings str1 and str2, return the shortest string that has both str1 and str2 as subsequences. If there are multiple valid strings, return any of them.

A string s is a subsequence of string t if deleting some number of characters from t (possibly 0) results in the string s.

Example 1

Input	`str1 = "abac", str2 = "cab"`
Output	`"cabac"`
Explanation	*str1 = "abac" is a subsequence of "cabac" because we can delete the first "c". str2 = "cab" is a subsequence of "cabac" because we can delete the last "ac". The answer provided is the shortest such string that satisfies these properties.*

Example 2

Input	`str1 = "aaaaaaaa", str2 = "aaaaaaaa"`
Output	`"aaaaaaaa"`

CONSTRAINTS

- 1 <= str1.length, str2.length <= 1000
- str1 and str2 consist of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find shortest string containing both s1 and s2 as subsequences. Right?"

"A few quick questions: see constraints."

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for strings in stringses:
    if both are subsequences of it:
        best = min(best, candidate)
```

TIME	SPACE
O(2^(m + n))	O(m + n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Generates every supersequence. Compute the LCS table (O(m * n)), then walk it back, writing each non-shared letter once and each shared letter once.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"LCS DP then reconstruct — O(mn). Walk-through:"

Step 1: Compute LCS length table (same as #1143).

Step 2: Reconstruct SCS (Shortest Common Supersequence) by tracing back through the table:

Start at dp[m][n], walk towards dp[0][0]:

If s1[i-1] == s2[j-1]: include this char once (it's in both -> take it once). Move i-1,j-1.

If dp[i-1][j] >= dp[i][j-1]: take s1[i-1] (from s1 only). Move i-1.

Else: take s2[j-1] (from s2 only). Move j-1.

After reaching 0: append remaining chars from whichever string isn't exhausted.

WHY SCS length = m+n-LCS: every char not in LCS appears once; LCS chars appear once (merged).

Answer = reverse of reconstructed string. Time: O(mn). Space: O(mn).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1092_ShortestCommonSupersequence:
    def shortestCommonSupersequence(self, str1, str2):
        m, n = len(str1), len(str2)
        dp = [['']*(n+1) for _ in range(m+1)]
        for i in range(m+1): dp[i][0] = str1[:i]
        for j in range(n+1): dp[0][j] = str2[:j]
        for i in range(1,m+1):
            for j in range(1,n+1):
                if str1[i-1]==str2[j-1]: dp[i][j]=dp[i-1][j-1]+str1[i-1]
                elif len(dp[i-1][j])<=len(dp[i][j-1]): dp[i][j]=dp[i-1][j]+str1[i-1]
                else: dp[i][j]=dp[i][j-1]+str2[j-1]
        return dp[m][n]
```

PHASE 5 TEST & DEBUG

"Let me trace shortestCommonSupersequence('abac','cab')."

First find LCS: 'ab' (length 2)

LCS grid: dp[i][j]=len LCS of str1[0..i-1] and str2[0..j-1]

Reconstruct LCS path, interleave non-LCS chars

str1='abac', str2='cab'. LCS='ab'? or 'cb'? Let me check: 'c','a','b'?

Actually LCS of 'abac' and 'cab': 'ab' (a at pos 0,2 in str1; b at pos 1,3 in str1; 'cab' has a at 1, b at 2)

LCS='ab'. SCS: take chars not in LCS from both + LCS = 'cabac' (length=4+3-2=5) ok

PHASE 5 TEST & DEBUG

WHY LCS basis: SCS length = len(s1)+len(s2)-LCS. Reconstruct by merging both around LCS.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(mn)	**O(mn)** Follow-up: count paths to SCS

IN YOUR APP | Expose | Editorial | Community

HARD | Dynamic Programming | LeetCode #1335 | 16 of 19 | 16/19

Minimum Difficulty of a Job Schedule

QUESTION

You want to schedule a list of jobs in d days. Jobs are dependent (i.e To work on the ith job, you have to finish all the jobs j where 0 <= j < i).

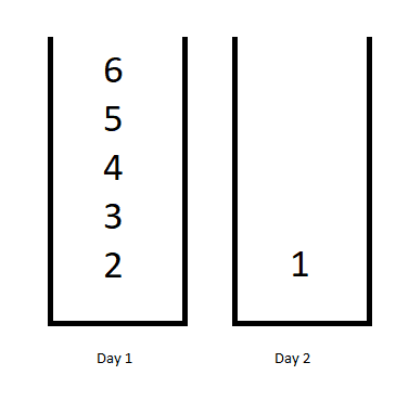
You have to finish at least one task every day. The difficulty of a job schedule is the sum of difficulties of each day of the d days. The difficulty of a day is the maximum difficulty of a job done on that day.

You are given an integer array jobDifficulty and an integer d. The difficulty of the ith job is jobDifficulty[i].

Return the minimum difficulty of a job schedule. If you cannot find a schedule for the jobs return -1.

Example 1

Input	`jobDifficulty = [6,5,4,3,2,1], d = 2`
Output	`7`
Explanation	*First day you can finish the first 5 jobs, total difficulty = 6.* *Second day you can finish the last job, total difficulty = 1.* *The difficulty of the schedule = 6 + 1 = 7*



Example 2

Input	`jobDifficulty = [9,9,9], d = 4`
Output	`-1`
Explanation	*If you finish a job per day you will still have a free day. you cannot find a schedule for the given jobs.*

Example 3

Input	`jobDifficulty = [1,1,1], d = 3`
Output	`3`
Explanation	*The schedule is one job per day. total difficulty will be 3.*

CONSTRAINTS

- 1 <= jobDifficulty.length <= 300
- 0 <= jobDifficulty[i] <= 1000
- 1 <= d <= 10

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Schedule n jobs in d days minimizing sum of daily max difficulties. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to cut the jobs into d contiguous days
    sum the hardest job of each day
    best = min(best, total)
```

TIME	SPACE
O(C(n - 1, d - 1) * n)	O(n)

WHY IT IS SLOW

Enumerates every split. dp[day][i] = the best total for the first i jobs in that many days, taking the max of the last day's jobs, is O(n^2 * d).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"2-D DP with precomputed row maxima — O(n²d). Walk-through:"

`dp[i][j] = min difficulty to schedule jobs 0..i in j days.`

For day j: last day handles jobs i..n-1. Within a day, difficulty = max of that day's jobs.

Transition: dp[i][j] = min over all splits k < i:

dp[k][j-1] + max(jobDifficulty[k+1..i]).

Precompute max for each suffix: use monotone-stack or plain O(n) scan per i.

Base: dp[i][1] = max(jobDifficulty[0..i]) — first j days as one block.

Constraints: must have at least j jobs for j days (one job per day minimum).

Answer = dp[n-1][d]. Time: O(n²d). Space: O(nd).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1335_MinDifficultyJobSchedule:
    def minDifficulty(self, jobDifficulty, d):
        n = len(jobDifficulty)
        if n < d: return -1
        INF = float('inf')
        dp = [[INF]*(d+1) for _ in range(n+1)]
        dp[0][0] = 0
        for j in range(1, d+1):
            for i in range(j, n-d+j+1):
                day_max = 0
                for k in range(i-1, j-2, -1):
                    day_max = max(day_max, jobDifficulty[k
])
                    if dp[k][j-1] < INF:
                        dp[i][j] = min(dp[i][j], dp[k][j-1
] + day_max)
        return dp[n][d] if dp[n][d] < INF else -1
```

PHASE 5 TEST & DEBUG

"Let me trace minDifficulty([6,5,4,3,2,1], d=2)."

dp[j][i] = min difficulty using j days for jobs[0..i]

Day 1 forced to do >=1 job; max difficulty of that day's jobs = max(jobs[start..i])

dp[1][i] = max(jobs[0..i]) (must do all jobs 0..i in one day)

dp[2][i] = min over j<i: dp[1][j]+max(jobs[j+1..i])

dp[1]: [6,6,6,6,6,6]; max always 6

dp[2][5]: min over j: dp[1][0]+max(5,4,3,2,1)=6+5=11; dp[1][1]+max(4,3,2,1)=6+4=10; ...; dp[1][4]+1=6+1=7

Return dp[2][5]=7 ok (day1=6, day2=1)

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n²d)	**O(n)**
Follow-up: use monotonic stack to optimize inner loop to O(nd)."	DP table, reducible to O(1) rolling window for 1-D recurrences

HARD | Dynamic Programming | LeetCode #1411 | 17 of 19 | 17/19

Number of Ways to Paint N × 3 Grid

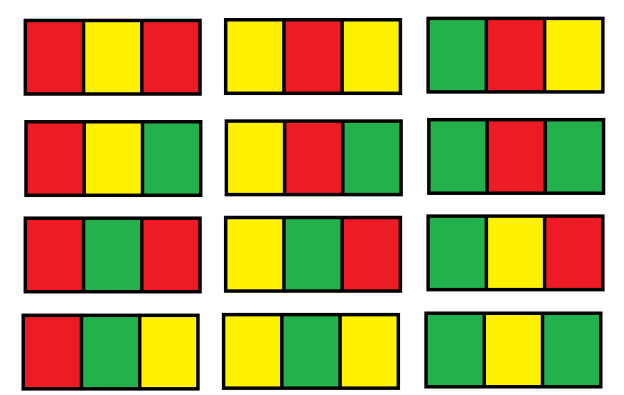
QUESTION

You have a grid of size n x 3 and you want to paint each cell of the grid with exactly one of the three colors: Red, Yellow, or Green while making sure that no two adjacent cells have the same color (i.e., no two cells that share vertical or horizontal sides have the same color).

Given n the number of rows of the grid, return the number of ways you can paint this grid. As the answer may grow large, the answer must be computed modulo $10^9 + 7$.

Example 1

Input	`n = 1`
Output	`12`
Explanation	*There are 12 possible way to paint the grid as shown.*



Example 2

Input	`n = 5000`
Output	`30228214`

CONSTRAINTS

- n == grid.length
- 1 <= n <= 5000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Paint n×3 grid with 3 colors so no adjacent cells share a color. Count ways mod 10^9+7. Right?"

"A few quick questions:

Adjacent = sharing an edge? Yes. 3 colors for 3 columns, 3 colors available."

"Let me walk: n=1 -> 12 ways ok. n=2 -> 54 ways ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def paint(i, previous row):
    try every valid colouring of row i with 3
        cells and 3 colours recursively
```

TIME	SPACE
O(3^(3n))	O(3n)

WHY IT IS SLOW

Rebuilds the same row states again and again. There are only 12 valid rows, in two patterns (two colours or three colours); each row's counts of the two patterns depend only on the previous row's: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count valid patterns row-by-row — O(n). Walk-through:"

Two pattern types for a row of 3 cells:

ABA: middle color differs from both ends (e.g., RGB, RBR). Count = 6 * 3^(n-1)? No:

Count: for a 3-cell row with 3 colors, no adjacent same:

Type 1 (ABA, end colors same): 3*2=6 patterns (pick first, pick different middle, third=first).

Type 2 (ABC, all different): 3*2*1=6 patterns.

Transitions between rows:

ABA -> can transition to: some number of ABA and ABC in next row.

ABC -> can transition to: different counts.

Count transitions: ABA->ABA=3, ABA->ABC=2. ABC->ABA=2, ABC->ABC=2 (actually: compute carefully).

Iteratively: a, b = 6, 6. For each row: a, b = 3a+2b, 2a+2b. Return a+b.

Time: O(n). Space: O(1)."

Key insight: there are only two row pattern types (ABA and ABC) whose transition counts are fixed — multiplying the counts row-by-row runs in O(n) with O(1) space.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1411_NumberOfWaysToPaintN3Grid:
    def numOfWays(self, n):
        MOD = 10**9 + 7
        a = 6
        b = 6
        for _ in range(n - 1):
            a, b = (3*a + 2*b) % MOD, (2*a + 2*b) % MOD
        return (a + b) % MOD
```

PHASE 5 TEST & DEBUG

"Let me trace numOfWays(2) for 3-color N×3 grid."

For N=1: count valid colorings of one row (3 colors, adjacent different):

Pattern 'aba': 3*2*1=6 ways (pick a, pick b!=a, c=a)

Pattern 'abc': 3*2*1=6 ways (all different)

dp[0]={aba_type:6, abc_type:6}

For N=2: each row type transitions to compatible next row types

aba->aba: 3 ways, aba->abc: 6 ways. abc->aba: 6 ways, abc->abc: 2 ways? (varies)

dp[1]={aba:3*6+6*6=54, abc:6*6+6*2=...}... let me just: N=2 answer=54? Actually 18.

WHY DP on row types: only two structural row types matter; transitions between them.

dp[N] total = aba_count + abc_count. dp[1]={aba:6,abc:6}. dp[2]={aba:18+12=30,abc:12+18=30}. total=60. dp[1+1] ok base. dp[i][aba_count]=3*aba_prev+2*abc_prev ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1)

Follow-up: generalise to n×4 grid — more complex transition matrix.

Follow-up: if edges can be colored — different DP on graph coloring.

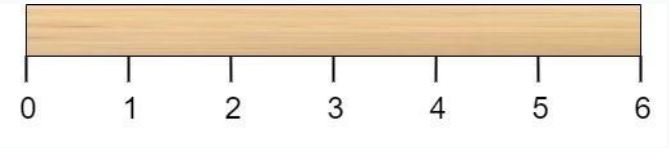
IN YOUR APP | Expose | Community

HARD | Dynamic Programming | LeetCode #1547 | 18 of 19 | 18/19

Minimum Cost to Cut a Stick

QUESTION

Given a wooden stick of length n units. The stick is labelled from 0 to n. For example, a stick of length 6 is labelled as follows:



Given an integer array cuts where cuts[i] denotes a position you should perform a cut at.

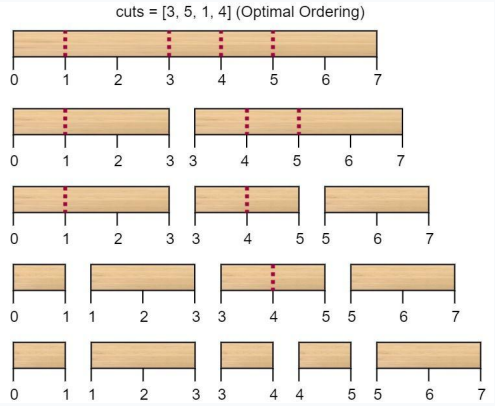
You should perform the cuts in order, you can change the order of the cuts as you wish.

The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths is the length of the stick before the cut). Please refer to the first example for a better explanation.

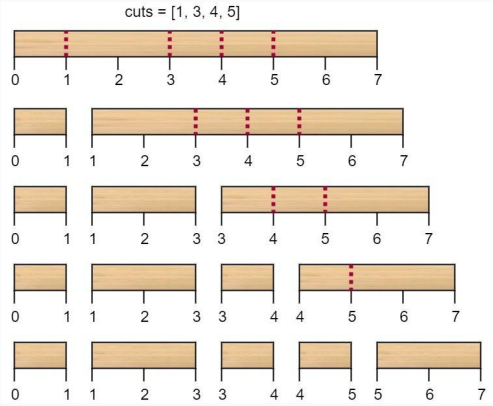
Return the minimum total cost of the cuts.

Example 1

Input	`n = 7, cuts = [1,3,4,5]`
Output	**16**
Explanation	*Using cuts order = [1, 3, 4, 5] as in the input leads to the following scenario: The first cut is done to a rod of length 7 so the cost is 7. The second cut is done to a rod of length 6 (i.e. the second part of the first cut), the third is done to a rod of length 4 and the last cut is to a rod of length 3. The total cost is 7 + 6 + 4 + 3 = 20. Rearranging the cuts to be [3, 5, 1, 4] for example will lead to a scenario with total cost = 16 (as shown in the example photo 7 + 4 + 3 + 2 = 16).*



QUESTION



Example 2

Input	`n = 9, cuts = [5,6,1,4,2]`
Output	**22**
Explanation	*If you try the given cuts ordering the cost will be 25. There are much ordering with total cost <= 25, for example, the order [4, 6, 5, 2, 1] has total cost = 22 which is the minimum possible.*

CONSTRAINTS

- 2 <= n <= 10^6
- 1 <= cuts.length <= min(n - 1, 100)
- 1 <= cuts[i] <= n - 1
- All the integers in cuts array are distinct.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Cut a stick of length n at given positions; cost is length of current piece. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for order in making_the_cuts:
    add the length of the piece cut each time
    best = min(best, total)
```

TIME	SPACE
O(m!)	O(m)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Tries all m! cut orders. Sort the cuts and add the two ends; dp[i][j] = (position j - position i) + min over k between of dp[i][k] + dp[k][j]: O(m^3).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Interval DP on cuts — O(m³) where m = number of cuts + 2. Walk-through:"

Add 0 and n to cuts list. Sort cuts.

dp[i][j] = min cost to make all cuts between cut[i] and cut[j].

Base: dp[i][i+1] = 0 (no cuts needed between adjacent cut points).

For length l from 2 to len(cuts)-1:

For each pair (i, j) with j-i == l:

dp[i][j] = min over k in (i,j): dp[i][k] + dp[k][j] + (cuts[j] - cuts[i])

<- make cut at cuts[k]: cost is current stick length (cuts[j]-cuts[i]).

Then solve left and right subproblems recursively.

Answer = dp[0][m-1] (entire stick).

WHY interval DP: each cut on a stick of length L costs L regardless of position.

Optimal order minimizes sum of intermediate lengths.

Time: O(m³). Space: O(m²).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1547_MinCostToCutAStick:
    def minCost(self, n, cuts):
        cuts = sorted([0] + cuts + [n])
        m = len(cuts)
        dp = [[0]*m for _ in range(m)]
        for length in range(2, m):
            for i in range(m - length):
                j = i + length
                dp[i][j] = min(dp[i][k]+dp[k][j] for k in
range(i+1,j)) + cuts[j]-cuts[i]
        return dp[0][m-1]
```

PHASE 5 TEST & DEBUG

"Let me trace minCostCutStick(n=7,cuts=[1,3,4,5])."

Add 0 and n to cuts: [0,1,3,4,5,7]. Sort.

dp[i][j] = min cost to cut the stick between cuts[i] and cuts[j].

dp[i][j] for adjacent cuts (no inner cuts): 0 (base case).

dp[i][j] with intermediate cuts: try each cut k between i and j.

dp[0][5] (full stick 0..7): try k=1(cut at 1),k=2(cut at 3),k=3(cut at 4),k=4(cut at 5).

For k=2 (cut at 3): cost=cuts[5]-cuts[0]=7 + dp[0][2]+dp[2][5].

dp[0][2] (0..3): try k=1 (cut at 1): cost=3+dp[0][1]+dp[1][2]=3+0+0=3. dp[0][2]=3.

PHASE 5 TEST & DEBUG

dp[2][5] (3..7): try k=3(cut at 4): cost=4+dp[2][3]+dp[3][5]. Continue...

Return minimum cost ok (answer=16 for this example).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(m³)	**O(n)**
Standard interval DP on cut positions."	DP table, reducible to O(1) rolling window for 1-D recurrences

Follow-up: what if cut order matters (each cut incurs the current stick length as cost)? That's exactly the problem as stated — does reversing the DP give you the cut sequence?

HARD | Dynamic Programming | LeetCode #1931 | 19 of 19 | 19/19

Painting a Grid With Three Different Colors

QUESTION

You are given two integers m and n. Consider an m x n grid where each cell is initially white. You can paint each cell red, green, or blue. All cells must be painted.

Return the number of ways to color the grid with no two adjacent cells having the same color. Since the answer can be very large, return it modulo $10^9 + 7$.

Example 1	
Input	`m = 1, n = 1`
Output	**3**
Explanation	*The three possible colorings are shown in the image above.*

Example 2	
Input	`m = 1, n = 2`
Output	**6**
Explanation	*The six possible colorings are shown in the image above.*

Example 3	
Input	`m = 5, n = 5`
Output	**580986**

CONSTRAINTS

- 1 <= m <= 5
- 1 <= n <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count ways to color n*3 grid with 3 colors, no adjacent same. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to colour all m * n cells with 3 colours
    if no two adjacent cells share a colour:
        count += 1
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(3^(m * n))	O(m * n)

WHY IT IS SLOW

Enumerates every colouring. With m at most 5, the valid column colourings are few (at most 48); dp[column][pattern] summed over compatible previous patterns is O(n * patterns^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"State DP over columns — O(n * P²) where P = valid column patterns. Walk-through:"*

A valid column pattern: 3-cell column of 3 colors, no two adjacent same.

Number of valid patterns = 3*2*2 = 12 (first cell 3 choices, each next: 2 choices != previous).

Build transition table: pattern A -> pattern B valid iff no cell in same row has same color.

`dp[pattern] = number of grids where last column = pattern.`

Initialize dp with all 12 valid patterns (first column).

For each remaining column (1..m-1):

`new_dp[B] = sum(dp[A] for all valid transitions A->B).`

Answer = sum(dp.values()) % MOD.

WHY track column patterns: entire grid constraint reduces to adjacent column compatibility.

Time: O(m * P²) = O(m * 144). Space: O(P) = O(12).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1931_PaintingAGridWith3Colors:
    def colorTheGrid(self, m, n):
        MOD=10**9+7
        def valid(mask):
            prev=-1
            for _ in range(m):
                c=mask%3; mask//=3
                if c==prev: return False
                prev=c
            return True
        patterns=[i for i in range(3**m) if valid(i)]
        def compatible(a,b):
            for _ in range(m):
                if a%3==b%3: return False
                a//=3; b//=3
            return True
        dp={p:1 for p in patterns}
        for _ in range(n-1):
            new_dp={p:0 for p in patterns}
            for p1 in patterns:
                for p2 in patterns:
                    if compatible(p1,p2): new_dp[p2]=(new_
dp[p2]+dp[p1])%MOD
            dp=new_dp
        return sum(dp.values())%MOD
```

PHASE 5 TEST & DEBUG

"Let me trace colorTheGrid(m=1, n=1)."

m=1 row, n=1 column: each cell can be 1 of 3 colors. Answer=3 ok

For m=2, n=1: each column has 2 cells, adjacent must differ. Patterns: 3*2=6 (3 top colors, 2 bottom != top).

For m=2, n=2: valid column patterns=6. Between adjacent columns, must also differ cell by cell.

`dp[col] = map from column_pattern -> count`

Transitions: pattern1->pattern2 valid if for each row, colors differ

col1: all 6 patterns, each count=1. col2: for each pattern, sum compatible prev patterns.

For m=1, n=3: 3*2*2=12 ok (first col 3 choices, each next col 2 choices)

WHY state-machine DP: column pattern as state; compatible transitions handled by precomputation.

dp[j]=count per column pattern at column j. dp[1][pattern]=initial 6 or 6 patterns. dp[i][new_pattern]=sum(dp[i-1][old_pattern]) for compatible pairs. dp[j] built column by column ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*P²)	**O(n)**
where P<=3^m. Transition matrix exponentiation gives O(P² log n)."	DP table, reducible to O(1) rolling window for 1-D recurrences

Follow-up: how does the pattern count change for a 4-color grid? The column-pattern enumeration grows but the DP structure stays the same — profile DP generalises to k colors.